Representation Learning for Rankings

Lecture 5: Dense Retrieval — From Bi-Encoders to Late Interaction

Avishek Anand

Information Retrieval

February 14, 2026

Outline

1. Recap & Motivation — From Re-ranking to Retrieval
2. Contrastive Learning for Retrieval
3. Dense Passage Retrieval (DPR)
4. ColBERT: Late Interaction
5. ANCE: Dynamic Hard Negatives
6. Training Dense Retrievers in Practice
7. Theoretical Foundations
8. Evaluation & The Zero-Shot Challenge
9. Hands-On with PyTerrier
10. Connections and Summary
11. Summary and Assignment

Section 1

Recap & Motivation — From Re-ranking to Retrieval

Recap: Where We Are

What We've Built So Far

- **Lecture 2:** Inverted indexes and ANN indexes (IVF+PQ, HNSW)
- **Lecture 3:** BM25, Language Models, RM3 for scoring and expansion
- **Lecture 4:** BERT cross-encoders for re-ranking (monoBERT, monoT5)

The Remaining Problem

Cross-encoders are **too slow for retrieval**:

- Re-ranking 100 docs: 0.5s ✓ Full corpus (8.8M): 12+ hours ✗
- BM25 misses semantic matches: "automobile issues" vs. "car problems"
- Recall ceiling: if BM25 misses a relevant doc, the cross-encoder can **never** recover it

Today's Goal

The Missing Piece

Combine the **semantic understanding** of BERT with the **retrieval speed** of ANN indexes from Lecture 2.

Key Idea: The Bi-Encoder

1. Encode all documents **offline** with BERT $\rightarrow$ dense vectors
2. Store in ANN index (IVF+PQ from Lecture 2!)
3. At query time: encode query with BERT $\rightarrow$ ANN search $\rightarrow$ top-k

Today's Goal

The Missing Piece

Combine the **semantic understanding** of BERT with the **retrieval speed** of ANN indexes from Lecture 2.

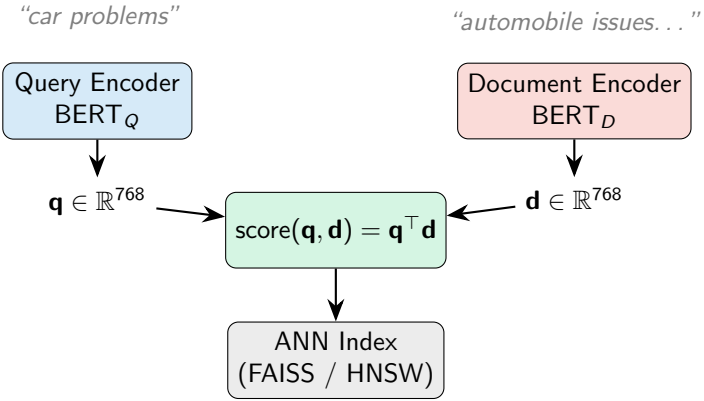
Key Idea: The Bi-Encoder

1. Encode all documents **offline** with BERT $\rightarrow$ dense vectors
2. Store in ANN index (IVF+PQ from Lecture 2!)
3. At query time: encode query with BERT $\rightarrow$ ANN search $\rightarrow$ top-k

The Trade-off

Bi-encoders are **less accurate** than cross-encoders (no query-document interaction) but **fast enough for full-corpus retrieval**.

The Dense Retrieval Pipeline



Offline (once):

- Encode all 8.8M passages
- Build ANN index

Online (per query):

- Encode query: $\sim$5 ms
- ANN search: $\sim$10 ms

Cross-Encoder vs. Bi-Encoder

Cross-Encoder: BERT([CLS] query [SEP] doc [SEP])

$\rightarrow$ relevance score

Bi-Encoder: $\text{BERT}_Q(\text{query})$ $\text{BERT}_D(\text{doc})$

$\rightarrow \mathbf{q}^\top \mathbf{d}$

	Cross-Encoder	**Bi-Encoder**
Query-doc interaction	Full (all layers)	None (dot product only)
Pre-compute doc embeddings	✗ No	✓ Yes (offline)
Use for retrieval	✗ No (too slow)	✓ Yes (with ANN)
Accuracy	Higher	Lower
Speed at inference	∼5 ms/pair	∼15 ms total

Section 2

Contrastive Learning for Retrieval

Contrastive Learning: Core Principle

Objective

Learn embeddings such that:

$$\text{score}(\mathbf{q}, \mathbf{d}^+) \gg \text{score}(\mathbf{q}, \mathbf{d}^-)$$

for relevant pair $(\mathbf{q}, \mathbf{d}^+)$ and irrelevant pair $(\mathbf{q}, \mathbf{d}^-)$.

Contrastive Learning: Core Principle

Objective

Learn embeddings such that:

$$\text{score}(\mathbf{q}, \mathbf{d}^+) \gg \text{score}(\mathbf{q}, \mathbf{d}^-)$$

for relevant pair $(\mathbf{q}, \mathbf{d}^+)$ and irrelevant pair $(\mathbf{q}, \mathbf{d}^-)$.

Intuition: The embedding space is a **map of meaning**.

- "car problems" and "automobile issues" $\rightarrow$ nearby points
- "car problems" and "car racing" $\rightarrow$ far apart

Contrastive Learning: Core Principle

Objective

Learn embeddings such that:

$$\text{score}(\mathbf{q}, \mathbf{d}^+) \gg \text{score}(\mathbf{q}, \mathbf{d}^-)$$

for relevant pair $(\mathbf{q}, \mathbf{d}^+)$ and irrelevant pair $(\mathbf{q}, \mathbf{d}^-)$.

Intuition: The embedding space is a **map of meaning**.

- "car problems" and "automobile issues" $\rightarrow$ nearby points
- "car problems" and "car racing" $\rightarrow$ far apart

Connection to Metric Learning

Define distance $d(\mathbf{q}, \mathbf{d}) = 1 - \cos(\mathbf{q}, \mathbf{d})$ and require:

$$d(\mathbf{q}, \mathbf{d}^+) + \text{margin} < d(\mathbf{q}, \mathbf{d}^-)$$

The InfoNCE Loss

InfoNCE / In-Batch Contrastive Loss

Given query $\mathbf{q}_i$, positive passage $\mathbf{p}_i^+$, and $B-1$ in-batch negatives:

$$\mathcal{L} = -\log \frac{\exp\left(\mathbf{q}_i^\top \mathbf{p}_i^+ / \tau\right)}{\sum_{j=1}^{B} \exp\left(\mathbf{q}_i^\top \mathbf{p}_j / \tau\right)}$$

The InfoNCE Loss

InfoNCE / In-Batch Contrastive Loss

Given query $\mathbf{q}_i$, positive passage $\mathbf{p}_i^+$, and $B-1$ in-batch negatives:

$$\mathcal{L} = -\log \frac{\exp\left(\mathbf{q}_i^\top \mathbf{p}_i^+ / \tau\right)}{\sum_{j=1}^{B} \exp\left(\mathbf{q}_i^\top \mathbf{p}_j / \tau\right)}$$

Key components:

- B: batch size — provides $B-1$ **free** negatives per query
- τ: temperature — controls sharpness of the softmax
- This is B-way classification: "which passage is relevant?"

The InfoNCE Loss

InfoNCE / In-Batch Contrastive Loss

Given query $\mathbf{q}_i$, positive passage $\mathbf{p}_i^+$, and $B-1$ in-batch negatives:

$$\mathcal{L} = -\log \frac{\exp(\mathbf{q}_i^\top \mathbf{p}_i^+ / \tau)}{\sum_{j=1}^{B} \exp(\mathbf{q}_i^\top \mathbf{p}_j / \tau)}$$

Key components:

- B: batch size — provides $B-1$ **free** negatives per query
- τ: temperature — controls sharpness of the softmax
- This is B-way classification: "which passage is relevant?"

No Negative Sampling Needed!

Worked Example: InfoNCE Computation (1/2)

Batch of 3 queries, each with 1 positive passage ($\tau = 1$):

	$\mathbf{p}_1$ ("aspirin effects")	$\mathbf{p}_2$ ("Tesla founding")	$\mathbf{p}_3$ ("Python loops")
$\mathbf{q}_1$ ("aspirin side effects")	**0.9** ✓	0.1	0.2
$\mathbf{q}_2$ ("who founded Tesla")	0.15	**0.85** ✓	0.1
$\mathbf{q}_3$ ("for loops in Python")	0.05	0.1	**0.8** ✓

Similarity matrix (dot products between query and passage embeddings).

Worked Example: InfoNCE Computation (1/2)

Batch of 3 queries, each with 1 positive passage ($\tau = 1$):

	$\mathbf{p}_1$ ("aspirin effects")	$\mathbf{p}_2$ ("Tesla founding")	$\mathbf{p}_3$ ("Python loops")
$\mathbf{q}_1$ ("aspirin side effects")	**0.9** ✓	0.1	0.2
$\mathbf{q}_2$ ("who founded Tesla")	0.15	**0.85** ✓	0.1
$\mathbf{q}_3$ ("for loops in Python")	0.05	0.1	**0.8** ✓

Similarity matrix (dot products between query and passage embeddings).

Goal: Diagonal entries (positives) should dominate each row.

Worked Example: InfoNCE Computation (2/2)

For query $\mathbf{q}_1$ ("aspirin side effects"):

Step 1: Exponentiate ($\tau = 1$):

$$e^{0.9} = 2.46, \quad e^{0.1} = 1.11, \quad e^{0.2} = 1.22$$

Worked Example: InfoNCE Computation (2/2)

For query $\mathbf{q}_1$ ("aspirin side effects"):

Step 1: Exponentiate ($\tau = 1$):

$$e^{0.9} = 2.46, \quad e^{0.1} = 1.11, \quad e^{0.2} = 1.22$$

Step 2: Softmax (normalize):

$$P(\mathbf{p}_1|\mathbf{q}_1) = \frac{2.46}{2.46 + 1.11 + 1.22} = \frac{2.46}{4.79} = \mathbf{0.51}$$

Worked Example: InfoNCE Computation (2/2)

For query $\mathbf{q}_1$ ("aspirin side effects"):

Step 1: Exponentiate ($\tau = 1$):

$$e^{0.9} = 2.46, \quad e^{0.1} = 1.11, \quad e^{0.2} = 1.22$$

Step 2: Softmax (normalize):

$$P(\mathbf{p}_1|\mathbf{q}_1) = \frac{2.46}{2.46 + 1.11 + 1.22} = \frac{2.46}{4.79} = \mathbf{0.51}$$

Step 3: Loss (negative log):

$$\mathcal{L}_1 = -\log(0.51) = 0.67$$

Interpretation

The model assigns 51% probability to the correct passage. Training will push this higher by increasing $\mathbf{q}_1^\top \mathbf{p}_1$ and decreasing $\mathbf{q}_1^\top \mathbf{p}_2$, $\mathbf{q}_1^\top \mathbf{p}_3$

Temperature Scaling: Why It Matters

$$p_j = \frac{\exp(s_j/\tau)}{\sum_k \exp(s_k/\tau)}$$

Small τ (e.g., 0.07):

- Distribution becomes **sharp**
- Strong gradients
- Risk: training instability
- Used by: DPR

Large τ (e.g., 1.0):

- Distribution becomes **smooth**
- Weaker gradients, more stable
- Risk: slow convergence
- Used by: ColBERT

Temperature Scaling: Why It Matters

$$p_j = \frac{\exp(s_j/\tau)}{\sum_k \exp(s_k/\tau)}$$

Small τ (e.g., 0.07):

- Distribution becomes **sharp**
- Strong gradients
- Risk: training instability
- Used by: DPR

Large τ (e.g., 1.0):

- Distribution becomes **smooth**
- Weaker gradients, more stable
- Risk: slow convergence
- Used by: ColBERT

Gradient Magnitude

$\|\nabla\mathcal{L}\| \propto 1/\tau$. Smaller temperature $\rightarrow$ stronger signal but training can diverge. This is a critical hyperparameter.

Triplet Loss: The Alternative

Triplet Margin Loss

$$\mathcal{L}_{\text{triplet}} = \max\big(0,\ d(\mathbf{q}, \mathbf{d}^+) - d(\mathbf{q}, \mathbf{d}^-) + m\big)$$

✓ **Advantages:**

- Explicit margin m enforces separation
- Easy to implement
- Fast convergence with hard triplets

✗ **Disadvantages:**

- Only **one** negative per anchor
- Margin m is a sensitive hyperparameter
- Wastes batch information

Triplet Loss: The Alternative

Triplet Margin Loss

$$\mathcal{L}_{\text{triplet}} = \max\big(0,\ d(\mathbf{q}, \mathbf{d}^+) - d(\mathbf{q}, \mathbf{d}^-) + m\big)$$

✓ **Advantages:**

- Explicit margin m enforces separation
- Easy to implement
- Fast convergence with hard triplets

✗ **Disadvantages:**

- Only **one** negative per anchor
- Margin m is a sensitive hyperparameter
- Wastes batch information

In Practice

InfoNCE dominates because it uses **all** $B - 1$ batch passages as negatives simultaneously — much richer training signal per step.

Section 3

Dense Passage Retrieval (DPR)

DPR: Architecture (Karpukhin et al., 2020)

"who founded wikipedia"

BERT_Q

$\mathbf{q} = \texttt{[CLS]}_Q \in \mathbb{R}^{768}$

"Wikipedia was founded by..."

BERT_D

$\mathbf{p} = \texttt{[CLS]}_D \in \mathbb{R}^{768}$

Key Design Choices

- **Separate encoders:** $\text{BERT}_Q \neq \text{BERT}_D$ (different weights, can specialize)
- **Representation:** [CLS] token output — single vector per text
- **Similarity:** $\text{score}(\mathbf{q}, \mathbf{p}) = \mathbf{q}^\top \mathbf{p}$ (dot product)
- **No interaction:** Query and passage never "see" each other during encoding

DPR: Training Recipe

Loss: InfoNCE with in-batch negatives

$$\mathcal{L} = -\log \frac{e^{\mathbf{q}_i^\top \mathbf{p}_i^+/\tau}}{\sum_{j=1}^{B} e^{\mathbf{q}_i^\top \mathbf{p}_j/\tau}}$$

Negatives per query:

1. $B - 1$ in-batch negatives (free!)
2. 1 BM25 hard negative (precomputed)

Param	**Value**
Batch size	128
Temperature τ	1.0
Learning rate	10^{-5}
Epochs	40
Optimizer	Adam
Max $\|q\|$	64 tokens
Max $\|p\|$	256 tokens

DPR: Training Recipe

Loss: InfoNCE with in-batch negatives

$$\mathcal{L} = -\log \frac{e^{\mathbf{q}_i^\top \mathbf{p}_i^+/\tau}}{\sum_{j=1}^{B} e^{\mathbf{q}_i^\top \mathbf{p}_j/\tau}}$$

Negatives per query:

1. $B-1$ in-batch negatives (free!)
2. 1 BM25 hard negative (precomputed)

Param	Value
Batch size	128
Temperature τ	1.0
Learning rate	10^{-5}
Epochs	40
Optimizer	Adam
Max $\lvert q \rvert$	64 tokens
Max $\lvert p \rvert$	256 tokens

Key Insight

Mixing **one** BM25 hard negative with in-batch random negatives is surprisingly effective. The BM25 negative is lexically similar but not relevant — forces the model beyond surface matching.

DPR: Why BM25 Hard Negatives?

Query: "side effects of aspirin"

Random negative: "The capital of France is Paris. . ."

- $\mathbf{q}^\top \mathbf{d}^- \approx 0.05$ — trivially easy, **no learning signal**

DPR: Why BM25 Hard Negatives?

Query: "side effects of aspirin"

Random negative: "The capital of France is Paris..."

- $\mathbf{q}^\top \mathbf{d}^- \approx 0.05$ — trivially easy, **no learning signal**

BM25 hard negative: "Aspirin was invented by Bayer in 1899..."

- Contains "aspirin" $\rightarrow$ BM25 ranks it high
- But **not about side effects** $\rightarrow$ not relevant
- $\mathbf{q}^\top \mathbf{d}^- \approx 0.6$ — confusing, **strong learning signal!**

DPR: Why BM25 Hard Negatives?

Query: "side effects of aspirin"

Random negative: "The capital of France is Paris..."

- $\mathbf{q}^\top \mathbf{d}^- \approx 0.05$ — trivially easy, **no learning signal**

BM25 hard negative: "Aspirin was invented by Bayer in 1899..."

- Contains "aspirin" $\rightarrow$ BM25 ranks it high
- But **not about side effects** $\rightarrow$ not relevant
- $\mathbf{q}^\top \mathbf{d}^- \approx 0.6$ — confusing, **strong learning signal!**

The Principle

The model must learn: "just because a passage mentions aspirin doesn't mean it's about side effects." This is **exactly** what BM25 cannot do.

DPR: Results & Significance

Method	NQ Top-20 Recall	MS MARCO MRR@10	Params
BM25	59.1	18.7	—
DPR	**78.4**	**31.1**	220M

Why DPR Matters

- First system to conclusively show **dense** > **sparse** for open-domain QA
- +19pp top-20 recall over BM25 on Natural Questions
- Enabled the modern "retriever + reader" paradigm

DPR: Results & Significance

Method	NQ Top-20 Recall	MS MARCO MRR@10	Params
BM25	59.1	18.7	—
DPR	**78.4**	**31.1**	220M

Why DPR Matters

- First system to conclusively show **dense** > **sparse** for open-domain QA
- +19pp top-20 recall over BM25 on Natural Questions
- Enabled the modern "retriever + reader" paradigm

But...

A single 768-dim vector for the entire document? That's a severe **information bottleneck**. Can we do better?

Section 4

ColBERT: Late Interaction

The Single-Vector Bottleneck

Problem with DPR

The **entire** document is compressed into a single 768-dim vector.

"Wikipedia was founded by Jimmy Wales and Larry Sanger in January 2001"

↓ information bottleneck

$\mathbf{d} \in \mathbb{R}^{768}$

The Single-Vector Bottleneck

Problem with DPR

The **entire** document is compressed into a single 768-dim vector.

"Wikipedia was founded by Jimmy Wales and Larry Sanger in January 2001"

↓ information bottleneck

$\mathbf{d} \in \mathbb{R}^{768}$

Consequences:

- Cannot capture fine-grained token-level alignments
- "**Who** founded Wikipedia?" vs. "**When** was Wikipedia founded?" $\rightarrow$ similar scores!
- Long documents lose information

The Single-Vector Bottleneck

Problem with DPR

The **entire** document is compressed into a single 768-dim vector.

"Wikipedia was founded by Jimmy Wales and Larry Sanger in January 2001"

↓ information bottleneck

$\mathbf{d} \in \mathbb{R}^{768}$

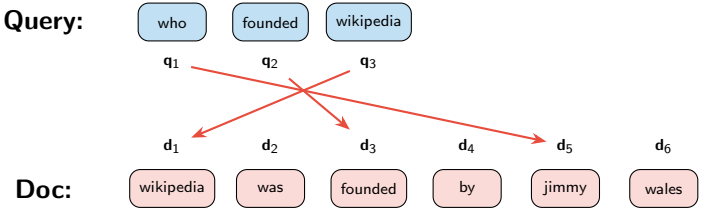
Consequences:

- Cannot capture fine-grained token-level alignments
- "**Who** founded Wikipedia?" vs. "**When** was Wikipedia founded?" $\rightarrow$ similar scores!
- Long documents lose information

ColBERT: Architecture (Khattab & Zaharia, 2020)

Key Idea

Encode query and document **separately** (like bi-encoder), but compare at the **token level** using MaxSim.

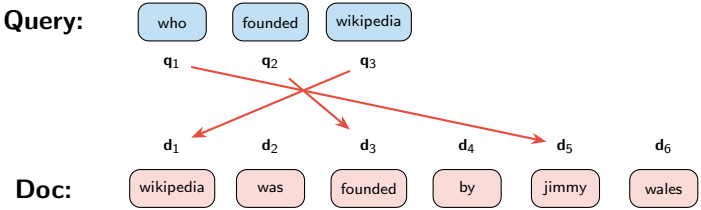


For each $\mathbf{q}_i$:
find best $\mathbf{d}_j$

ColBERT: Architecture (Khattab & Zaharia, 2020)

Key Idea

Encode query and document **separately** (like bi-encoder), but compare at the **token level** using MaxSim.



For each $\mathbf{q}_i$:
find best $\mathbf{d}_j$

Late Interaction

Documents are encoded **offline** (token-level embeddings stored). The "interaction" happens **late** — only at query time via MaxSim.

ColBERT: MaxSim Scoring

MaxSim Definition

$$\text{MaxSim}(\mathbf{q}, \mathbf{d}) = \sum_{i=1}^{|\mathbf{q}|} \max_{j=1}^{|\mathbf{d}|} \cos(\mathbf{q}_i, \mathbf{d}_j)$$

Step by step:

1. For each query token $\mathbf{q}_i$, find the document token $\mathbf{d}_j$ with highest cosine similarity
2. Take that maximum similarity
3. Sum across all query tokens

ColBERT: MaxSim Scoring

MaxSim Definition

$$\text{MaxSim}(\mathbf{q}, \mathbf{d}) = \sum_{i=1}^{|\mathbf{q}|} \max_{j=1}^{|\mathbf{d}|} \cos(\mathbf{q}_i, \mathbf{d}_j)$$

Step by step:

1. For each query token $\mathbf{q}_i$, find the document token $\mathbf{d}_j$ with highest cosine similarity
2. Take that maximum similarity
3. Sum across all query tokens

Properties:

- **Asymmetric:** Every query token must find a match; document tokens can be ignored
- **Decomposable:** Document embeddings precomputed and stored offline
- Token embeddings: $\mathbf{q}_i, \mathbf{d}_j \in \mathbb{R}^{128}$ (projected down from 768)

Worked Example: MaxSim Scoring

Query: "who founded wikipedia" (3 tokens)
Document: "wikipedia was founded by jimmy wales" (6 tokens)

	wikipedia	*was*	*founded*	*by*	*jimmy*	*wales*
who	0.12	0.08	0.15	0.10	**0.72**	0.45
founded	0.20	0.05	**0.95**	0.10	0.12	0.08
wikipedia	**0.98**	0.06	0.18	0.04	0.10	0.15

Worked Example: MaxSim Scoring

Query: "who founded wikipedia" (3 tokens)
Document: "wikipedia was founded by jimmy wales" (6 tokens)

	wikipedia	*was*	*founded*	*by*	*jimmy*	*wales*
who	0.12	0.08	0.15	0.10	**0.72**	0.45
founded	0.20	0.05	**0.95**	0.10	0.12	0.08
wikipedia	**0.98**	0.06	0.18	0.04	0.10	0.15

$$\text{MaxSim} = \underbrace{0.72}_{\text{who}\to\text{jimmy}} + \underbrace{0.95}_{\text{founded}\to\text{founded}} + \underbrace{0.98}_{\text{wikipedia}\to\text{wikipedia}} = \mathbf{2.65}$$

Notice

"who" aligns with "jimmy" (the answer entity) — the model learns **semantic alignment**, not just exact matching.

ColBERT: Efficiency vs. Accuracy

	DPR (Bi-Enc)	**ColBERT**	**Cross-Encoder**
Interaction	None (dot prod)	Late (MaxSim)	Full (cross-attn)
Pre-compute docs	✓ (1 vec)	✓ (n vecs)	✗
Use for retrieval	✓ With ANN	✓ With ANN	✗ Too slow
Speedup vs. CE	$\sim$1000$\times$	$\sim$100$\times$	1$\times$
MS MARCO MRR@10	31.1	36.0	39.0
Storage per doc	$768 \times 4\text{B} = 3$ KB	$128 \times 128 \times 4\text{B} \approx 64$ KB	N/A

ColBERT: Efficiency vs. Accuracy

	DPR (Bi-Enc)	ColBERT	Cross-Encoder
Interaction	None (dot prod)	Late (MaxSim)	Full (cross-attn)
Pre-compute docs	✓ (1 vec)	✓ (n vecs)	✗
Use for retrieval	✓ With ANN	✓ With ANN	✗ Too slow
Speedup vs. CE	$\sim$1000$\times$	$\sim$100$\times$	1$\times$
MS MARCO MRR@10	31.1	36.0	39.0
Storage per doc	$768 \times 4\text{B} = 3$ KB	$128 \times 128 \times 4\text{B} \approx 64$ KB	N/A

Sweet Spot

ColBERT gives near-cross-encoder accuracy at bi-encoder-like speed. Trade-off: **much higher storage** (one vector per token, not per document).

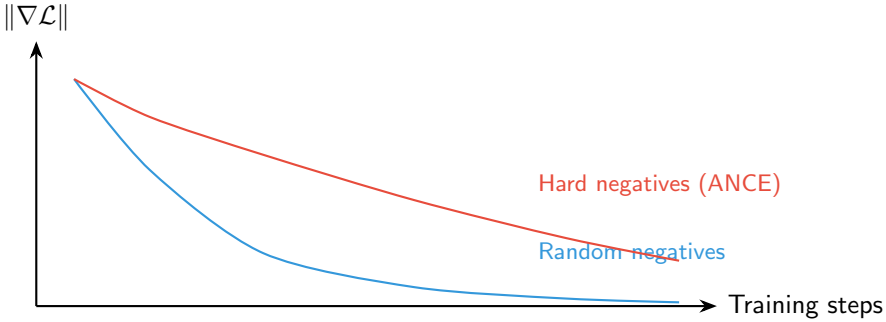
Section 5

ANCE: Dynamic Hard Negatives

The Hard Negative Problem (Xiong et al., 2021)

Observation

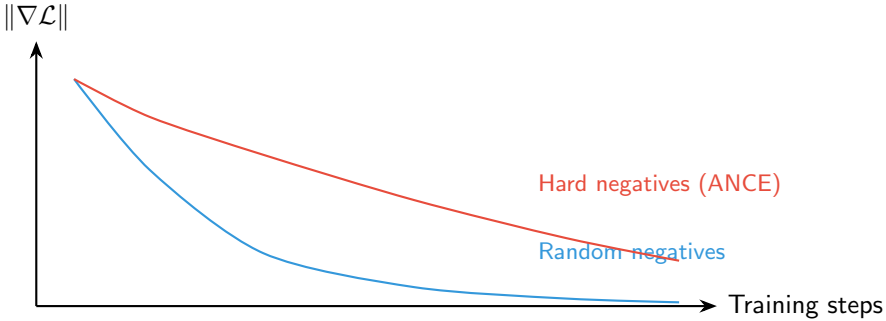
Most in-batch negatives are **trivially easy**. The model separates them instantly and stops learning.



The Hard Negative Problem (Xiong et al., 2021)

Observation

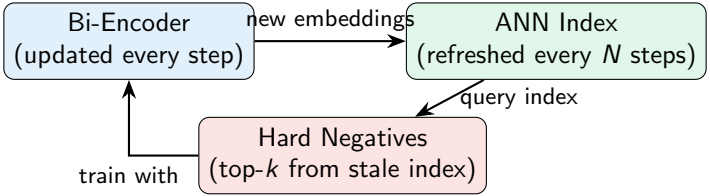
Most in-batch negatives are **trivially easy**. The model separates them instantly and stops learning.



ANCE: Asynchronous Index Refresh

Core Innovation

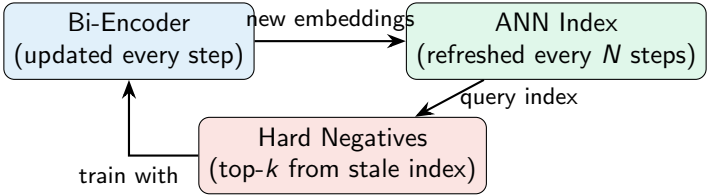
Maintain an ANN index during training. Use the model's **own** top-k nearest passages as hard negatives.



ANCE: Asynchronous Index Refresh

Core Innovation

Maintain an ANN index during training. Use the model's **own** top-k nearest passages as hard negatives.



Why the Lag is a Feature

The index lags by N steps. Negatives are hard (from a recent model) but not **impossibly** hard (from the current model). This stabilizes training.

ANCE: Gradient Analysis

Why Hard Negatives Produce Better Gradients

The gradient of InfoNCE w.r.t. $\mathbf{q}$ decomposes as:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{q}} = \underbrace{(p^+ - 1) \cdot \mathbf{p}^+}_{\text{pull toward positive}} + \underbrace{\sum_{j \neq +} p_j \cdot \mathbf{p}_j}_{\text{push from negatives}}$$

where $p_j = \text{softmax}(\mathbf{q}^\top \mathbf{p}_j / \tau)$.

ANCE: Gradient Analysis

Why Hard Negatives Produce Better Gradients

The gradient of InfoNCE w.r.t. $\mathbf{q}$ decomposes as:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{q}} = \underbrace{(p^+ - 1) \cdot \mathbf{p}^+}_{\text{pull toward positive}} + \underbrace{\sum_{j \neq +} p_j \cdot \mathbf{p}_j}_{\text{push from negatives}}$$

where $p_j = \text{softmax}(\mathbf{q}^\top \mathbf{p}_j / \tau)$.

Easy negatives:

- $p_j \approx 0$ for all $j \neq +$
- Push terms **vanish**
- No learning signal

Hard negatives:

- $p_j > 0$ for confusing j
- Push terms are **large**
- Strong, informative gradient

ANCE: Results

Method	NQ Top-20 R	MS MARCO MRR@10	Negatives
BM25	59.1	18.7	—
DPR (BM25 neg.)	78.4	31.1	Static / lexical
ANCE	**81.9**	**33.0**	Dynamic / semantic

ANCE: Results

Method	NQ Top-20 R	MS MARCO MRR@10	Negatives
BM25	59.1	18.7	—
DPR (BM25 neg.)	78.4	31.1	Static / lexical
ANCE	**81.9**	**33.0**	Dynamic / semantic

Key Takeaway

The **only** difference between DPR and ANCE is the negative sampling strategy. Semantic hard negatives from the model's own index are strictly better than BM25 hard negatives.

Implication

Negative sampling is a **first-order concern** in dense retrieval. The model, the loss,

Section 6

Training Dense Retrievers in Practice

Negative Sampling Strategies

The Core Question

Query has 1 labeled positive. Where do the negatives come from?

1. **Random negatives**
   - Sample random passage from corpus
   - Easy to distinguish $\rightarrow$ weak training signal
   - ✗ Not recommended for production

Negative Sampling Strategies

The Core Question

Query has 1 labeled positive. Where do the negatives come from?

1. **Random negatives**
   - Sample random passage from corpus
   - Easy to distinguish $\rightarrow$ weak training signal
   - ✗ Not recommended for production
2. **BM25 hard negatives** ✓ *Good starting point*
   - Top-k BM25 results that are *not* labeled relevant
   - Lexically similar but semantically different $\rightarrow$ strong signal

Negative Sampling Strategies

The Core Question

Query has 1 labeled positive. Where do the negatives come from?

1. **Random negatives**
   - Sample random passage from corpus
   - Easy to distinguish → weak training signal
   - ✗ Not recommended for production
2. **BM25 hard negatives** ✓ *Good starting point*
   - Top-k BM25 results that are *not* labeled relevant
   - Lexically similar but semantically different → strong signal
3. **ANN hard negatives (ANCE)** ✓ *Best quality*
   - Top-k from model's own index (refreshed periodically)
   - Semantically hard — forces fine-grained distinctions

The False Negative Problem

Issue

In-batch negatives may actually be **relevant** (but unlabeled).

Example: Query "Who founded Wikipedia?"

- **Labeled positive:** "Wikipedia was founded by Jimmy Wales and Larry Sanger..."
- **In-batch "negative":** "Jimmy Wales co-founded Wikipedia in 2001..."
  ← Actually relevant! But treated as negative.

The False Negative Problem

Issue

In-batch negatives may actually be **relevant** (but unlabeled).

Example: Query "Who founded Wikipedia?"

- **Labeled positive:** "Wikipedia was founded by Jimmy Wales and Larry Sanger..."
- **In-batch "negative":** "Jimmy Wales co-founded Wikipedia in 2001..."
  ← Actually relevant! But treated as negative.

Consequence: Model penalized for correctly ranking a relevant passage high.

Mitigations

- Confidence regularization: down-weight uncertain negatives
- Larger corpora reduce false negative probability
- Cross-encoder denoising (re-score negatives, remove false ones)

Training Hyperparameters

Batching & Memory

- Max sequence: 256 tokens (passage)
- Batch size: 128 (critical!)
- **Larger batch = more negatives = better**
- Gradient accumulation to simulate larger batches
- Mixed precision (fp16) saves $\sim$50% memory

Optimization

- Learning rate: 1–3 $\times$ 10^{-5}
- Warmup: 10% of steps
- Scheduler: linear decay
- Epochs: 20–40
- Temperature τ: 0.05–1.0
- Evaluate every 5K steps

Training Hyperparameters

Batching & Memory

- Max sequence: 256 tokens (passage)
- Batch size: 128 (critical!)
- **Larger batch = more negatives = better**
- Gradient accumulation to simulate larger batches
- Mixed precision (fp16) saves ∼50% memory

Optimization

- Learning rate: $1\text{–}3 \times 10^{-5}$
- Warmup: 10% of steps
- Scheduler: linear decay
- Epochs: 20–40
- Temperature τ: 0.05–1.0
- Evaluate every 5K steps

Key Difference from Cross-Encoder Training

Batch size matters **much more** for bi-encoders. Each additional batch element provides a free negative. Going from batch 32 to 128 can improve MRR by 2–3 points.

Common Training Mistakes

Avoid These Pitfalls

1. **Batch size too small**
   - Batch 16 = only 15 negatives per query
   - Use gradient accumulation to simulate larger batches
2. **Only random negatives**
   - Model never sees hard cases $\rightarrow$ weak representations
   - Always include BM25 or ANN hard negatives
3. **Wrong temperature**
   - Too small: training diverges
   - Too large: convergence too slow
   - Start with $\tau = 0.05$, tune on dev set
4. **Not normalizing embeddings**
   - Without L2 normalization, embedding magnitudes grow unbounded
   - Always normalize before computing similarity

Training Code Sketch (Hugging Face + Sentence-Transformers)

```python
from sentence_transformers import SentenceTransformer
from sentence_transformers import InputExample, losses

# Load pre-trained bi-encoder
model = SentenceTransformer('bert-base-uncased')

# Create training pairs
train_examples = [
    InputExample(texts=[query, pos_passage, neg_passage])
    for query, pos_passage, neg_passage in triples
]

# MultipleNegativesRankingLoss = InfoNCE
train_loss = losses.MultipleNegativesRankingLoss(model)

# Train
model.fit(
    train_objectives=[(train_dataloader, train_loss)],
```

Section 7

Theoretical Foundations

Why Does Contrastive Learning Work?

InfoNCE as Mutual Information Lower Bound (Oord et al., 2018)

$$I(\mathbf{q}; \mathbf{d}^+) \geq \log(B) - \mathcal{L}_{\text{InfoNCE}}$$

Interpretation: Minimizing InfoNCE **maximizes** a lower bound on mutual information between query and positive document embeddings.

Why Does Contrastive Learning Work?

InfoNCE as Mutual Information Lower Bound (Oord et al., 2018)

$$I(\mathbf{q}; \mathbf{d}^+) \geq \log(B) - \mathcal{L}_{\text{InfoNCE}}$$

Interpretation: Minimizing InfoNCE **maximizes** a lower bound on mutual information between query and positive document embeddings.

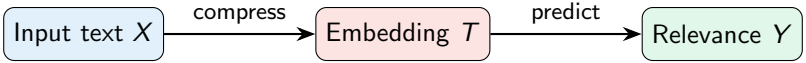
Why This Matters for Retrieval

- High $I(\mathbf{q}; \mathbf{d}^+)$: query embedding is **predictive** of relevant documents
- The bound is tighter with more negatives (larger B)
- This explains why **large batch sizes help**: tighter MI bound

The Information Bottleneck Perspective

IB Objective (Tishby et al., 2000)

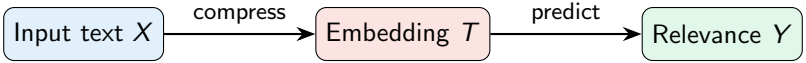
$$\min_{T} \left[\beta \cdot I(T;X) - I(T;Y)\right]$$



The Information Bottleneck Perspective

IB Objective (Tishby et al., 2000)

$$\min_{T} \left[\beta \cdot I(T;X) - I(T;Y)\right]$$



- $I(T;Y)$ **high**: Embedding retains what's needed for relevance
- $I(T;X)$ **low**: Embedding discards noise and irrelevant features

Implication

Dense retrieval generalizes because the bottleneck forces the model to learn **essential semantic features**, not surface patterns.

Representation Collapse

Definition

All embeddings converge to a single point (or low-rank subspace):

$$\forall\, i, j: \quad \|\mathbf{d}_i - \mathbf{d}_j\| \to 0 \quad \Rightarrow \quad \text{score}(\mathbf{q}, \mathbf{d}_i) \approx \text{score}(\mathbf{q}, \mathbf{d}_j)$$

Representation Collapse

Definition

All embeddings converge to a single point (or low-rank subspace):

$$\forall\, i, j: \quad \|\mathbf{d}_i - \mathbf{d}_j\| \to 0 \quad \Rightarrow \quad \text{score}(\mathbf{q}, \mathbf{d}_i) \approx \text{score}(\mathbf{q}, \mathbf{d}_j)$$

Two collapse modes:

1. **Complete collapse:** All embeddings map to $\mathbf{0}$
   Cause: no negatives $\Rightarrow$ trivial zero-loss solution
2. **Dimensional collapse:** Embeddings use only a few dimensions
   Cause: easy negatives $\Rightarrow$ only coarse features needed

Representation Collapse

Definition

All embeddings converge to a single point (or low-rank subspace):

$$\forall\, i, j: \quad \|\mathbf{d}_i - \mathbf{d}_j\| \to 0 \quad \Rightarrow \quad \text{score}(\mathbf{q}, \mathbf{d}_i) \approx \text{score}(\mathbf{q}, \mathbf{d}_j)$$

Two collapse modes:

1. **Complete collapse:** All embeddings map to $\mathbf{0}$
   Cause: no negatives $\Rightarrow$ trivial zero-loss solution
2. **Dimensional collapse:** Embeddings use only a few dimensions
   Cause: easy negatives $\Rightarrow$ only coarse features needed

Prevention

Hard negatives + ℓ_2 normalization + proper temperature. Hard negatives force the model to use all dimensions $\to$ prevents collapse.

Neural Collapse & Optimal Geometry (Papyan et al., 2020)

Equiangular Tight Frame (ETF)

In the terminal phase of training, class means $\{\boldsymbol{\mu}_c\}$ form a maximally symmetric structure:

$$\boldsymbol{\mu}_i^\top \boldsymbol{\mu}_j = \begin{cases} 1 & i = j \\ -\frac{1}{C-1} & i \neq j \end{cases}$$

Properties:

- All class centers are **equidistant** (maximally separated)
- Points collapse to their class mean (minimal within-class variance)
- **Maximum margin** between all classes

Neural Collapse & Optimal Geometry (Papyan et al., 2020)

Equiangular Tight Frame (ETF)

In the terminal phase of training, class means $\{\boldsymbol{\mu}_c\}$ form a maximally symmetric structure:

$$\boldsymbol{\mu}_i^\top \boldsymbol{\mu}_j = \begin{cases} 1 & i = j \\ -\frac{1}{C-1} & i \neq j \end{cases}$$

Properties:

- All class centers are **equidistant** (maximally separated)
- Points collapse to their class mean (minimal within-class variance)
- **Maximum margin** between all classes

Relevance to Retrieval

Hard negatives push representations toward ETF geometry. Without them, dimensional collapse prevents this optimal structure from forming.

Section 8

Evaluation & The Zero-Shot Challenge

Key Benchmarks

Benchmark	Corpus	Queries	Primary Metric
MS MARCO	8.8M passages	500K train	MRR@10
Natural Questions	21M passages	3.5K test	Top-20 Recall
BEIR (18 tasks)	Varies	Varies	nDCG@10 (zero-shot)
TREC-DL 2019/20	8.8M passages	43/54	nDCG@10

Key Benchmarks

Benchmark	Corpus	Queries	Primary Metric
MS MARCO	8.8M passages	500K train	MRR@10
Natural Questions	21M passages	3.5K test	Top-20 Recall
BEIR (18 tasks)	Varies	Varies	nDCG@10 (zero-shot)
TREC-DL 2019/20	8.8M passages	43/54	nDCG@10

Critical Distinction

- MS MARCO / NQ: **In-domain** (train & test same distribution)
- BEIR: **Zero-shot** (train on MS MARCO, test on 18 diverse tasks)
- Dense methods excel in-domain but often struggle on BEIR!

The Zero-Shot Challenge: BEIR

	BM25	**DPR**	**ColBERT**	**ANCE**	**Contriever**
BEIR Avg nDCG@10	47.3	33.2	41.5	37.7	48.5

The Zero-Shot Challenge: BEIR

	BM25	DPR	ColBERT	ANCE	Contriever
BEIR Avg nDCG@10	47.3	33.2	41.5	37.7	48.5

Surprising Finding

BM25 outperforms most dense models on zero-shot BEIR!

Why?

- Dense models overfit to NQ/MS MARCO query structure
- BM25 is distribution-agnostic (just uses term statistics)
- BEIR includes fact-checking, citation prediction, argument retrieval

The Zero-Shot Challenge: BEIR

	BM25	DPR	ColBERT	ANCE	Contriever
BEIR Avg nDCG@10	47.3	33.2	41.5	37.7	48.5

Surprising Finding

BM25 outperforms most dense models on zero-shot BEIR!

Why?

- Dense models overfit to NQ/MS MARCO query structure
- BM25 is distribution-agnostic (just uses term statistics)
- BEIR includes fact-checking, citation prediction, argument retrieval

Solution: Contriever (Izacard et al., 2022)

Unsupervised contrastive pre-training on diverse data **before** fine-tuning. Achieves 48.5 — finally beating BM25 on BEIR.

Hybrid Retrieval: Dense + Sparse

Key Finding

Combining BM25 and dense retrieval **outperforms either alone**.

$$\text{score}_{\text{hybrid}}(q, d) = \alpha \cdot \text{score}_{\text{BM25}}(q, d) + (1 - \alpha) \cdot \text{score}_{\text{dense}}(q, d)$$

Method	NQ Top-20 Recall
BM25 alone	62.9
DPR alone	78.4
BM25 + DPR (hybrid)	**82.6**

Hybrid Retrieval: Dense + Sparse

Key Finding

Combining BM25 and dense retrieval **outperforms either alone**.

$$\text{score}_{\text{hybrid}}(q, d) = \alpha \cdot \text{score}_{\text{BM25}}(q, d) + (1 - \alpha) \cdot \text{score}_{\text{dense}}(q, d)$$

Method	NQ Top-20 Recall
BM25 alone	62.9
DPR alone	78.4
BM25 + DPR (hybrid)	**82.6**

Why Hybrid Works

Sparse captures **exact matches** (named entities, rare terms). Dense captures

Section 9

Hands-On with PyTerrier

Setup: Dense Retrieval with PyTerrier

```python
import pyterrier as pt
if not pt.started():
    pt.init()

dataset = pt.get_dataset('irds:msmarco-passage/dev/small')
topics = dataset.get_topics().head(50)
qrels = dataset.get_qrels()

# BM25 baseline
index = pt.IndexFactory.of("./msmarco_index")
bm25 = pt.BatchRetrieve(index, wmodel="BM25")

# Dense retrieval with ANCE
from pyterrier_dr import FlexIndex, TctColBert

# Load pre-trained dense encoder
encoder = TctColBert()  # or ANCE, DPR, etc.
flex_index = FlexIndex("./dense_index")
```

Exercise 1: BM25 vs. Dense Retrieval

```
# Compare BM25 vs. Dense vs. Hybrid
hybrid = (bm25 % 1000) | (dense % 1000)  # union

comparison = pt.Experiment(
    [bm25, dense, hybrid],
    topics, qrels,
    eval_metrics=["map", "ndcg_cut_10", "recall_100"],
    names=["BM25", "Dense (ANCE)", "Hybrid"]
)
print(comparison)
```

Questions to explore:

- Which queries does dense retrieval improve on?
- Which queries does BM25 still win?
- How much does hybrid add over each individual method?

Exercise 2: Dense + Cross-Encoder Pipeline

```
from pyterrier_t5 import MonoT5ReRanker

monoT5 = MonoT5ReRanker(model='castorini/monot5-base-msmarco')

# Three-stage pipeline: Dense -> Cross-Encoder
pipeline_dense_ce = (dense % 100) >> monoT5

# Compare all variants
results = pt.Experiment(
    [bm25,
     (bm25 % 100) >> monoT5,        # L4 pipeline
     dense,
     pipeline_dense_ce,               # New: dense + CE
     (hybrid % 100) >> monoT5],      # Best of all worlds
    topics, qrels,
    eval_metrics=["ndcg_cut_10", "recall_100"],
    names=["BM25", "BM25+monoT5", "Dense",
           "Dense+monoT5", "Hybrid+monoT5"]
```

Student Exercises

Exercise A: Semantic vs. Lexical Analysis (10 min)

- Find a query where dense retrieval finds relevant docs that BM25 missed
- Inspect: is it vocabulary mismatch? Paraphrase? Different phrasing?
- Compare the top-5 results from each

Exercise B: When Dense Fails (10 min)

- Find a query where BM25 beats dense retrieval
- Check: is it an entity name? Rare technical term? Exact match needed?
- Would hybrid retrieval fix it?

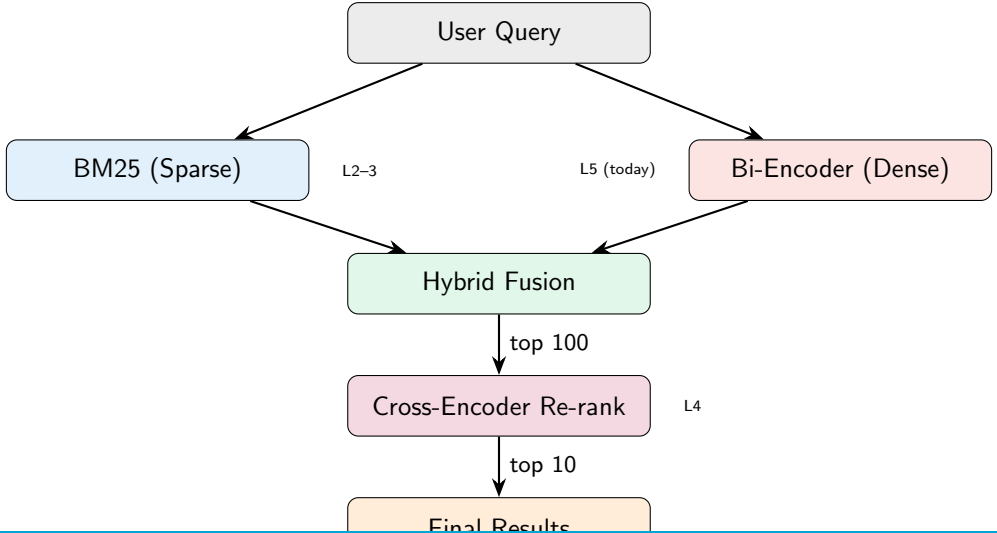
Exercise C: Recall Ceiling (5 min)

- Compare Recall@1000 of BM25 vs. dense vs. hybrid
- Which gives the best ceiling for the cross-encoder re-ranker?

Section 10

Connections and Summary

The Complete Modern Retrieval Stack



Preview: Lecture 6 — Generative Retrieval

A Completely Different Paradigm

What if the model **generates** the document identifier directly from the query? No index, no embeddings — just a seq2seq model.

What You'll Learn

- DSI: Differentiable Search Index (Tay et al., 2022)
- GENRE: Autoregressive Entity Retrieval (De Cao et al., 2021)
- SEAL: N-gram based document identifiers
- How to design document IDs: atomic, hierarchical, text-based
- When generative retrieval beats dense retrieval (and when it doesn't)

Preparation

Read: Tay et al. (2022), "Transformer Memory as a Differentiable Search Index"

Section 11

Summary and Assignment

Key Takeaways

1. Bi-Encoders Enable Semantic Retrieval at Scale

Encode documents offline, search with ANN indexes from Lecture 2. Solves the vocabulary mismatch problem that BM25 cannot.

2. Contrastive Learning Is the Training Paradigm

InfoNCE loss with in-batch negatives. Larger batches = more negatives = better training. Temperature τ is a critical hyperparameter.

3. DPR $\rightarrow$ ColBERT $\rightarrow$ ANCE: Improving Dense Retrieval

DPR: single-vector bi-encoder. ColBERT: token-level late interaction. ANCE: dynamic hard negatives via asynchronous index refresh.

4. Hybrid Retrieval Is the Practical Standard

Dense + sparse complementary. BM25 for exact match, dense for semantics. Cross-encoder on top for final precision.

Assignment

Continues from Lecture 4 assignment. Additional tasks:

Part 7: Dense Retrieval (30 pts)

- Implement dense retrieval pipeline (DPR or TCT-ColBERT)
- Compare against BM25 baseline on the same queries
- Build hybrid pipeline (BM25 + dense)
- Report Recall@100, MRR@10, and NDCG@10

Part 8: Analysis (20 pts)

- Identify 3 queries where dense beats BM25 (explain why: vocabulary mismatch?)
- Identify 2 queries where BM25 beats dense (explain why: entity matching?)
- Compare the full pipeline: hybrid + monoT5 vs. BM25 + monoT5
- Discuss: does the hybrid first stage improve the cross-encoder ceiling?

Questions?

Questions?

Office Hours: TBD

Email: avishekanand@gmail.com

Key formula to remember:

$$\mathcal{L}_{\text{InfoNCE}} = -\log \frac{\exp(\mathbf{q}^\top \mathbf{p}^+/\tau)}{\sum_i \exp(\mathbf{q}^\top \mathbf{p}_i/\tau)}$$

Backup: Sentence-BERT (Reimers & Gurevych, 2019)

Key Contribution

First to show that fine-tuning BERT as a **Siamese network** produces high-quality sentence embeddings. Predates DPR.

Differences from DPR:

- **Shared** encoder (same weights for query and passage)
- Trained on NLI data (natural language inference), not IR data
- Originally for sentence similarity, not retrieval
- Good general-purpose embeddings, but DPR better for open-domain QA

Legacy

SBERT established the bi-encoder paradigm. DPR adapted it specifically for passage retrieval with better negatives and separate encoders.

Backup: Batch Size Effects

Why does batch size matter so much for InfoNCE?

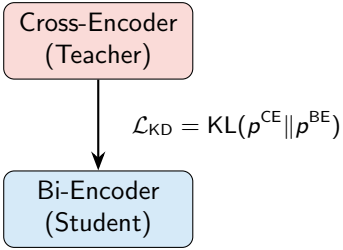
Batch Size	Negatives/Query	MI Bound	Quality
32	31	$\leq \log(32) = 3.5$	Moderate
128	127	$\leq \log(128) = 4.9$	Good
512	511	$\leq \log(512) = 6.2$	Excellent

Theoretical Justification

$I(\mathbf{q}; \mathbf{d}^+) \geq \log(B) - \mathcal{L}$. Larger B gives a tighter bound on mutual information. More negatives = richer gradient signal = better representations.

Practical tip: Use gradient accumulation if GPU memory is limited. 4 steps $\times$ batch 32 $\approx$ effective batch 128.

Backup: Knowledge Distillation from Cross-Encoders



Process

1. Train cross-encoder on MS MARCO (high accuracy)
2. Use cross-encoder to score top-k BM25 passages for each query
3. Train bi-encoder to **mimic** cross-encoder's ranking (soft labels)

Result: Bi-encoder approaches cross-encoder quality at bi-encoder speed. This is how production systems like TAS-B are trained.

Backup: Open Research Frontiers

1. **Closing the BEIR gap**
   - Contriever, RetroMAE: unsupervised pre-training
   - E5, GTE: instruction-tuned retrievers
2. **Scaling to billions of documents**
   - Binary quantization, product quantization
   - Matryoshka representations (flexible dimensionality)
3. **Multimodal retrieval**
   - CLIP for image–text retrieval
   - ColPali: late interaction for document images
4. **Integration with LLMs**
   - Retrieval-augmented generation (RAG)
   - LLM-based re-ranking (RankGPT from Lecture 4 backup)

References

- Karpukhin et al. (2020). Dense Passage Retrieval for Open-Domain QA. *EMNLP*.
- Khattab & Zaharia (2020). ColBERT: Efficient and Effective Passage Search. *SIGIR*.
- Xiong et al. (2021). Approximate Nearest Neighbor Negative Contrastive Learning for Dense Text Retrieval. *ICLR*.
- Reimers & Gurevych (2019). Sentence-BERT. *EMNLP*.
- Oord et al. (2018). Representation Learning with Contrastive Predictive Coding. *arXiv*.
- Izacard et al. (2022). Unsupervised Dense Information Retrieval with Contrastive Learning. *TMLR*.
- Thakur et al. (2021). BEIR: A Heterogeneous Benchmark for Zero-shot IR. *NeurIPS Datasets*.
- Papyan et al. (2020). Prevalence of Neural Collapse. *PNAS*.
- Tishby et al. (2000). The Information Bottleneck Method. *Allerton*.
- Hofstätter et al. (2021). Efficiently Teaching an Effective Dense Retriever. *SIGIR*.